

Neuro-symbolic learning over OWL 2 DL via consequence-based compilation to differentiable circuits

Olga Mashkova¹[0000–0002–4916–1660], Asaad
Mohammedsaleh¹[0009–0007–3160–8819], Fernando
Zhapa-Camacho¹[0000–0002–0710–2259] and Robert
Hoehndorf¹[0000–0001–8149–5890]

Computer Science Program, Computer, Electrical, and Mathematical Sciences &
Engineering Division, King Abdullah University of Science and Technology, Thuwal
23955, Saudi Arabia
`{first.name.last.name}@kaust.edu.sa`

Abstract. OWL 2 DL ontologies, grounded in the description logic *SRQIQ*, express large knowledge bases in biomedicine and the Semantic Web. Neuro-symbolic (NeSy) learners over description logics either embed the ontology in a continuous space, abandoning classical entailment, or restrict to the Horn fragment $\mathcal{EL}^{++}$, which has a single canonical model. We present BAOBAB, which compiles a *SRQIQ* ontology with a finite ABox into a Sentential Decision Diagram (SDD): it saturates a propositional core under a consequence-based calculus and instantiates the remaining *SRQIQ* features (nominals, number restrictions, and the role axioms) over the active domain. The SDD’s evidence-conditioned weighted model count then trains a perception network to *recognize real images* under partial ABox supervision: on an ontology that exercises every distinctive *SRQIQ* feature, a CNN learns to read MNIST digits coupled by a successor relation and recovers latent ontology concepts that an independent perception leaves at chance. When the supervision admits several ontology-consistent completions, an independent perception collapses onto one, a *reasoning shortcut*: we show that a mixture indexed by the query’s justifications can represent the calibrated posterior no independent perception can, and that seeding it from the circuit’s enumerated completions attains the Bayes-optimal posterior on a real-image MNIST task where single-WMC and learned mixtures (the BEARS-ensemble hypothesis class) do not: to our knowledge the first to characterize and mitigate reasoning shortcuts in a non-Horn description logic. Soundness of the compiler and the representation result are machine-checked in Lean 4. Code is available at <https://github.com/bio-ontology-research-group/baobab>.

1 Introduction

OWL 2 DL ontologies, formally grounded in the description logic *SRQIQ* [11], structure knowledge bases in biomedicine, scientific cataloging, and the Semantic

Web. We ask a neural network to read raw inputs into the concepts of such an ontology when most of those concepts carry no direct labels, so the network must infer them from the ontology’s logical structure. The neuro-symbolic (NeSy) community has produced two broad families of OWL-aware learners. *Embedding-based* methods such as EL Embeddings [19], OWL2Vec* [6], BoxEL [36], Box²EL [13], and FALCON [31] project an ontology into a continuous space; they scale to OWL-shaped knowledge graphs but compute no classical entailment during training. *Knowledge-compilation* methods such as DeepProbLog [23], Semantic Loss [37], Semantic Probabilistic Layers [1], Scallop [21], and NeuPSL [27] compute the weighted model count (WMC) of the compiled constraint and feed it to perception as a differentiable signal, but target propositional formulas or Datalog-shaped programs; the few NeSy methods that take a description-logic ontology as input target $\mathcal{ALC}$ or the OWL 2 Horn profiles, compiling to a probabilistic circuit [20] or relaxing to fuzzy semantics [35,40].

The gap between the two families of methods is structural. Knowledge compilation keeps the classical semantics end to end (the WMC of Θ under p_θ is exactly the probability that a world drawn from the perception’s per-atom posterior *satisfies the ontology*); but the propositional reduction of $\mathcal{SROIQ}$ adds disjunction, classical negation, qualified cardinality, nominals, and the full family of role axioms, which a Horn-only saturation calculus does not handle easily. Embedding-based NeSy buys scaling but loses classical semantics: a trained embedding can satisfy its loss while violating an entailment of the ontology.

We present BAOBAB, a compiler that takes a $\mathcal{SROIQ}$ ontology and produces a Sentential Decision Diagram (SDD) [7]: a circuit that scores whether the perception network’s predictions stay logically consistent with the ontology. Its weighted model count is the training signal under partial ABox supervision, so that a real image encoder learns latent ontology concepts it is never directly shown.

Specifically, we make three contributions. **(i) $\mathcal{SROIQ}$ compilation.** BAOBAB accepts any OWL 2 DL ontology and compiles it to an SDD, soundly on the grounding-covered fragment, combining consequence-based saturation over a propositional core ($\mathcal{ALCHOQ}$) [32,33] with finite-domain grounding of the $\mathcal{SROIQ}$ -specific extensions (nominals, qualified cardinality, and the six role-axiom shapes $\mathcal{ALC}$ lacks); an OWL loader [22] maps every OWL 2 DL axiom kind to the compiler’s syntax (§3.1). **(ii) Latent concept learning of real images.** The SDD’s evidence-conditioned weighted model count trains a CNN to *recognize MNIST digit images* through the ontology: under partial ABox supervision the perception recovers latent concepts it is never directly shown (digit identity $0.25 \rightarrow 0.99$) and drives ontology violations toward zero, where an independent perception sits at chance (§3.2, §4.1); the same workflow drives a ResNet through a role-based fragment of the real Pizzaiolo OWL ontology, recovering its latent pizza classes. **(iii) Reasoning-shortcut awareness in $\mathcal{SROIQ}$.** When the supervision leaves several ontology-consistent completions, a single independent perception provably cannot place calibrated mass on them, but a mixture indexed by the query’s *justifications* can; realized as one network (*JustWMC*) and

seeded from the completions the circuit enumerates, it attains the Bayes-optimal calibrated posterior on a real-image MNIST task where single-WMC and learned mixtures (the BEARS-ensemble hypothesis class) do not (§4.2). A per-axiom comparison shows that 23% of the *SRIOQ* axioms leave DeepProbLog’s Horn fragment (§4.3); §3.3 states the formal guarantees behind the compiler and the representation result.

2 Background

A *SRIOQ* knowledge base $(\mathcal{T}, \mathcal{R}, \mathcal{A})$ has a *TBox* of general concept inclusions $C \sqsubseteq D$, an *ABox* of concept and role assertions $C(a)$, $R(a, b)$ with (in)equalities, and an *RBox* of role hierarchies $R \sqsubseteq S$, role chains $R_1 \circ \dots \circ R_k \sqsubseteq S$, and the eight role characteristics (transitivity, (a)symmetry, (ir)reflexivity, (inverse-)functionality, disjointness), over the standard syntax of [11]; App. A gives the full syntax (concepts close Boolean connectives, nominals, qualified number restrictions, and $\exists R.\text{Self}$) and the Tarskian semantics.

Consequence-based (CB) reasoning saturates a set of clauses under a fixed family of resolution-like inference rules and reads the entailed concept subsumptions off the resulting closure, rather than searching for a model as a tableau would. CB calculi were developed for *SRIQ* [3] and extended to *ALCHOIQ* [32,33], where clauses are grouped into *contexts* (clause sets anchored at a concept) between which consequences propagate; they underlie Sequoia [33] (*SRIOQ*) and ELK [14] (the Horn profile). The calculus is deterministic and model-free, so the closure is a fixed propositional object.

Knowledge compilation represents a propositional theory as a circuit on which queries become efficient. A Sentential Decision Diagram (SDD) [7] is a decomposable, deterministic circuit over a fixed *vtree* (a binary tree over the variables); its size is worst-case exponential in the theory’s treewidth but, once built, supports linear-time *weighted model counting*. For atom weights $w(a) \in [0, 1]$, the weighted model count of a theory φ is

$$\text{WMC}(\varphi \mid w) = \sum_{\alpha \models \varphi} \prod_{a: \alpha(a)=1} w(a) \prod_{a: \alpha(a)=0} (1 - w(a)), \quad (1)$$

the total weight of φ ’s models, evaluated on an SDD in one bottom-up pass over the circuit.

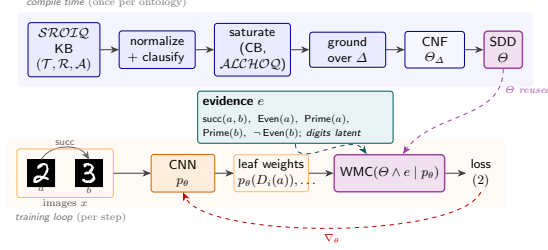
Reading each atom as an independent Bernoulli event with parameter $w(a)$ is the *distribution semantics* [30]: (1) is exactly the probability that a w -distributed world satisfies φ , the link knowledge-compilation neuro-symbolic methods exploit to turn per-atom probabilities into a differentiable training signal [37,23].

3 Methods

3.1 Knowledge compilation algorithm

The algorithm takes a *SRIOQ* knowledge base $(\mathcal{T}, \mathcal{R}, \mathcal{A})$ whose ABox ranges over a finite set of individuals Δ (the *active domain*), and returns an SDD

Fig. 1. The BAOBAB workflow. *Compile time* (once per ontology): the five stages above produce an SDD. *Training loop*: a shared CNN reads the coupled individuals a, b (real MNIST digits) into per-atom posteriors that weight the SDD leaves; the evidence-conditioned WMC is the differentiable loss.



over the ground propositional atoms $\{C(a), R(a, b) : a, b \in \Delta\}$, whose evidence-conditioned weighted model count is the differentiable training signal of §3.2. The algorithm runs in five stages: normalization, structural transformation to DL-clauses, consequence-based saturation over the propositional $\mathcal{ALCHOQ}$ sub-language, grounding of the $\mathcal{SRIOQ}$ -extension features over Δ , and SDD compilation of the resulting CNF (Figure 1).

Normalization rewrites every concept into negation-normal form (NNF) by the standard de Morgan, quantifier, and cardinality dualities (App. B). The *structural transformation* then turns the NNF axioms into DL-clauses $\bigwedge_i B_i \rightarrow \bigvee_j H_j$, whose body atoms B_i and head atoms H_j are concept atoms $C(t)$, role atoms $R(t, t')$, or equalities $t \approx t'$ over variables, individuals, and unary function terms $f(t)$, with a fresh name Q_D for each non-atomic subconcept D . Nominals and qualified number restrictions are not classified here but deferred to grounding (Table 2).

Saturation closes the DL-clause set under propositional *hyperresolution* (resolving body atoms against matching head atoms under a most-general unifier). Its role is to eliminate *Skolem terms*: head existentials and number restrictions classify to function terms $f(x)$ naming anonymous witnesses the finite grounder cannot instantiate, and saturation derives their *function-free* consequences over the named individuals. Four guards keep the closure terminating (tautology elimination, input-derived bounds on the variables and role atoms per clause, and a Skolem-term depth bound; exact bounds in App. C); the guarded calculus is sound (mechanized for the $\mathcal{ALCHOQ}$ core) but, being bounded, not complete.

Grounding instantiates the function-free clauses over the active domain Δ , replacing variables by individuals and expanding each deferred nominal, number restriction, and role characteristic into the propositional clauses listed below. The result is a propositional CNF Θ_Δ that is sound over the active domain, and whose models are *exactly* the $\mathcal{SRIOQ}$ interpretations over Δ when every existential consequence is grounding-covered or function-free (Theorem 3).

A construct’s consequences reach Θ_Δ either by *grounding* that closes the domain or by *saturation* that eliminates an existential witness symbolically; each deferred construct expands over Δ into boundedly many clauses justified by a soundness lemma (Theorem 3), and Table 2 lists every schema. App. F shows

why saturation is load-bearing exactly for ungrounded existentials and inert otherwise.

Finally, the grounded propositional CNF is compiled to an SDD with PySDD, and the differentiable WMC layer of [23] backpropagates the loss through the SDD’s parameterized leaves into the perception network (Algorithm 1 gives the end-to-end workflow).

3.2 Latent concept learning under partial ABox supervision

We address *ABox-supervised latent concept learning*. Each training instance supplies a perceptual input (an image, or a noisy feature vector) together with a partial set of ground ABox literals over an *observable* signature; the concept and role atoms the ontology entails over a separate *latent* signature are never directly supervised. The output is a per-atom posterior over the latent signature, scored against the generating interpretation; the latent atoms are tied to the observables only through $\mathcal{O}$, so the learner must propagate evidence through the ontology rather than read labels off the input.

Formally, let $\mathcal{O}$ be a fixed *SRQIQ* ontology, $p_\theta: \mathcal{A} \rightarrow [0, 1]$ the perception network’s per-atom posterior over the compiled circuit’s propositional signature $\mathcal{A}$, and $e \subseteq \mathcal{A} \times \{T, F\}$ a piece of supervisory evidence, a partial Boolean assignment over the example’s labels. The *consistent set* K_e is the set of complete assignments to $\mathcal{A}$ that satisfy $\Theta \wedge e$, where Θ is the SDD of $\mathcal{O}$. When the supervision underdetermines the latent identity, $|K_e| > 1$.

We optimize the perception network against the *evidence-conditioned weighted-model-count loss*

$$\mathcal{L}(p_\theta; e) = \text{BCE}(p_\theta(a), v) \Big|_{(a,v) \in e} + \lambda \cdot (-\log \text{WMC}(\Theta \wedge e \mid p_\theta)). \quad (2)$$

The label term is binary cross-entropy (BCE) directly supervising the observed labels; the semantic term is $-\log$ of the WMC of $\Theta \wedge e$ under p_θ , the probability that some completion satisfies the ontology *and* the evidence. When e determines a single model this collapses to the fully supervised likelihood; when $|K_e| > 1$ it distributes the gradient across K_e as a credal set, following [25,24]. Both terms are needed: label-only training ignores the ontology, while conditioning the semantic term on e (rather than on Θ alone) ties it to each example’s evidence instead of the ontology’s globally most probable assignment.

(2) still commits to a single mode of K_e when $|K_e| > 1$, and the reason is structural. A perception that scores each atom independently induces a product distribution, the *universal conditionally independent* (UCI) class $p_\mu^\perp = \prod_i \mu_i^{c_i} (1-\mu_i)^{1-c_i}$; the assignments such a distribution can concentrate on form a *subcube*: fix some atoms, let the rest vary freely. A *reasoning shortcut* (RS) is a latent assignment that satisfies the ontology yet differs from the one that generated the example [25]; with $|K_e| > 1$ the optimum of (2) over a product distribution is exactly such a shortcut. [17] show that an independent perception cannot reproduce the predictions of an RS mixture unless the mixture’s valid completions already form one such subcube: those consistent with a single

justification of the query (a minimal set of atom values forcing the query true). When the valid completions span several justifications, no independent perception can spread its mass over them; it collapses onto an arbitrary one. Theorem 5 establishes this characterization for *SRIOQ*.

The converse is previously known: a *mixture* of independent distributions can be RS-aware [17,24]. We adapt it to *SRIOQ*: a mixture of UCIs with one component per justification, $p_{\text{Just}} = \sum_k \pi_k(x) \prod_i \mu_{k,i}(x)^{c_i} (1 - \mu_{k,i}(x))^{1-c_i}$, represents every RS mixture (Theorem 5): it can place mass on completions drawn from different justifications, which no single product distribution can cover (the disjunction benchmark of §4.2 is the canonical case). We realize it as one network, the **MixtureEncoder**, and call the method *JustWMC*: a shared body feeds K atom heads μ_k and a softmax selector π , trained end-to-end on a mixture cross-entropy plus the selector-averaged semantic loss of (2), with a stop-gradient KL diversity term (after BEARS) that breaks head-permutation symmetry and spreads the heads across distinct Θ -consistent modes (App. G). BEARS trains K separate encoders and reports the best against an oracle; JustWMC returns one calibrated posterior with no oracle, its components readable off the circuit’s justifications (the anchored variant below) rather than trained.

The heads need not be learned: when the circuit enumerates the query’s justifications, we *seed* each head from one Θ -consistent completion (fixing its latent logits) and learn only π : *anchored* JustWMC, which realizes the positive direction of Theorem 5 by construction and reaches the calibrated posterior of §4.2 where the learned mixture, under the same objective, does not.

3.3 Formal guarantees

The compiler and its metatheory are formalized in Lean 4 [26]; the formal statements and proofs are in App. I and the module inventory in App. J. Four results support the claims above. *Saturation is sound* (Theorem 2): every derived clause is entailed, so over a fixed vtree the compiled SDD is invariant under saturation on the grounding-covered fragment (Theorem 1), and skipping it off that fragment over-approximates (sound but incomplete). *Each grounding rule is faithful* (Theorem 3): Θ_Δ is sound over Δ and, when every existential is grounding-covered or function-free, its models are exactly the *SRIOQ* interpretations over Δ . *The compiled SDD’s weighted model count* equals the probability the distribution semantics assigns (Theorem 4), the quantity the losses of §3.2 optimize. Finally, the RS-awareness characterization of [17] holds for *SRIOQ* (Theorem 5). The development also proves the *ALC* core sound and complete and bounds the grounding by a polynomial in $|\Delta|$.

4 Experiments

4.1 Real-image perception

Unlike prior knowledge-compilation methods for description logics, which report on hand-built feature vectors [20], we drive the circuit BAOBAB compiles with two

image encoders and recover ontology atoms that receive no direct supervision. All experiments use ten seeds; tables give mean $\pm$ std on held-out unseen individuals, and bold marks a one-sided paired permutation gain (Holm-corrected per table) at $p < 0.05$ (all reach $p < 0.01$).

Metrics. *Digit accuracy*: per-individual argmax over the five digit-identity atoms vs. the true digit (chance 0.20). *Per-atom accuracy (latent, property, topping)*: each named atom thresholded at 0.5, so the easy negative atoms put it above the argmax. *Violation*: fraction of examples whose MAP decode (latent atoms argmaxed, evidence clamped) has no model (WMC = 0). *Mode-coverage TV*: total variation to uniform over the Θ -consistent modes (0=perfect; one of M scores $1-1/M$). We also report standard ten-bin concept-marginal ECE [8] and the latent-atom NLL. Arrows in each table mark the improving direction.

MNIST-SROIQ (two supervision regimes). Each example pairs two real MNIST images a, b whose digits, restricted to 0–4, satisfy $\text{succ}(a, b)$ with $b = (a+1) \bmod 5$ (a restricted-digit task after rsbench [4]). One per-image CNN runs on both slots; a *SROIQ* ontology couples the pair through *every* distinctive *SROIQ* feature: universal and qualified-cardinality restrictions, the inverse and transitive succ with the chain $\text{succ} \circ \text{succ} \sqsubseteq \text{plusTwo}$ (compiled at $|\Delta| = 3$), complement, covering disjunctions, and the functional, (a)symmetric, and (ir)reflexive role characteristics; App. H lists it in full. The evidence mixes a *role* assertion ($\text{succ}(a, b)$) with *concept* assertions ($\text{Number}(a)$, $\text{Number}(b)$ and parity/primality); digit identities are never supervised. The amount of revealed concept evidence controls #RS, the number of Θ -consistent completions the evidence leaves open (the modes of K_e , §3.2; Table 1).

Grounded (succ , Number , and a random half of the parity/primality atoms; #RS = 1): the evidence pins the digits through the ontology, and WMC recovers the never-supervised identities ($0.25 \rightarrow 0.99$) at violation 0.02 and concept expected calibration error (ECE) 0.002; the ontology-blind independent perception stays at chance.

Under-determined (succ and Number only; #RS = 5): the universal/inverse/functional axioms admit the five cyclic relabelings of 0–4. The objective is invariant under them, so a factorized perception cannot pick one even when trained through the circuit: WMC reaches digit accuracy only 0.18 (chance 0.20) and leaves most decoded pairs successor-inconsistent (violation 0.78, concept ECE 0.15); the shortcut is intrinsic to the symmetry, not an optimization artifact. Only *grounding* breaks it: a single parity atom drops #RS to 3 and the full parity profile to 1, after which WMC recovers the digits and drives the violation to 0.02 (the grounded row).

Pizzaiolo (a real role-based OWL ontology). Each example is a synthetically rendered pizza image from the Pizzaiolo dataset [5], encoded by a frozen ImageNet ResNet-18 with a trainable linear head. The OWL ontology shipped with the dataset is genuinely role-based *SROIQ*: a *hasTopping* role links each pizza to its toppings and the pizza classes are defined by existential and universal restrictions

Table 1. MNIST-*SRQIQ*, mean $\pm$ std over ten seeds (metrics in §4; *digit* identities never supervised). Grounding the concept evidence collapses #RS from 5 to 1.

regime	method	digit	latent	viol $\downarrow$	ECE $\downarrow$
grounded (#RS=1)	Independent	0.25 $\pm$.09	0.49 $\pm$.14	0.80 $\pm$.12	0.33 $\pm$.14
	WMC	0.99$\pm$.00	1.00$\pm$.00	0.02$\pm$.01	0.00$\pm$.00
under-det. (#RS=5)	Independent	0.20 $\pm$.03	0.46 $\pm$.10	0.93 $\pm$.10	0.08 $\pm$.08
	WMC	0.18 $\pm$.07	0.67 $\pm$.03	0.78 $\pm$.05	0.15 $\pm$.02

over it (90 existential and 44 universal restrictions, 66 named classes). It *does not compile* to an SDD: it grounds in under a second, but the SDD exhausts memory even for a single pizza, a real-ontology instance of the circuit-size wall (§6; quantified in App. F, where grounding stays sub-second while the SDD exceeds 22 GB by three toppings).

To still drive the ResNet through the role structure we extract a compilable two-topping fragment (59 atoms): a `hasTopping` role to named topping individuals with the shipped definitions $\text{NonVeg} \equiv \text{Pizza} \sqcap \exists \text{hasTopping.Meat}$, $\text{Spicy} \equiv \text{Pizza} \sqcap \exists \text{hasTopping.Spicy}$, and $\text{Veg} \equiv \text{Pizza} \sqcap \neg \text{NonVeg}$. Compiling even this needs three sound, opt-in grounding refinements, documented and mechanized in App. D (the rest of the paper’s circuits stay byte-identical): role atoms are pruned to `hasTopping`’s declared domain and range; the cubic equality theory is skipped; and a closed qualified existential is materialized as the exact biconditional $X(a) \Leftrightarrow \bigsqcup_t \text{hasTopping}(a, t) \sqcap C(t)$ over the named toppings, recovering a class from the *absence* of a topping, not only its presence. The ResNet predicts the toppings (supervised) while the property classes are latent: WMC recovers them (property 0.45 $\rightarrow$ 0.92) and drives the violation from 0.89 to 0.06, where the independent perception stays at chance (Table 7, App. H).

The closed qualified-existential refinement is the one departure from open-world semantics in this fragment, so we ablate it directly (Table 3, App. D). Keeping the forward Skolemized direction but dropping the closure clause $\exists \text{hasTopping}.C \rightarrow X(a)$ leaves the supervised toppings untouched (0.93 $\rightarrow$ 0.95) but lowers latent property recovery from 0.92 to 0.78, which stays 0.33 above the independent baseline’s 0.45, so perception does not collapse; it becomes *over-permissive*. The reported violation falls to 0.00 because the removed clause is the constraint whose violation was counted: the open model satisfies a weaker theory vacuously rather than recovering the latents it can no longer identify. The closure direction therefore contributes a 14-point gain in latent recovery, which isolates it as a learning signal over and above the EL-style forward grounding. Without the closure clause, the *SRQIQ* fragment retains partial but degraded identifiability.

4.2 MNIST-Disjunction (RS-genuine, real images)

The MNIST regimes above are *over-determining* once a digit is grounded, so single-WMC and JustWMC coincide. We now build a task whose latents admit

several equally plausible Θ -consistent completions *per example*, on real images. Three individuals a, b, c each carry a real MNIST digit; a, b show consecutive digits coupled by $\text{succ}(a, b)$, and a shared CNN recovers them through a *digit* circuit (the grounded MNIST mechanism). On the same individuals an image-free *gender* attribute carries the reasoning shortcut: under $\text{Person} \sqsubseteq \text{Male} \sqcup \text{Female}$ (disjoint), $\text{Male} \sqsubseteq \forall \text{marriedTo.Female}$ with $\text{marriedTo}(a, b)$, and the parenthood roles (App. H), exactly four Θ -consistent gender modes remain (a, b opposite sex, c free): $\#RS = 4$. The pixels fix the digits but *not* the genders, so the gender modes are uninformed and uniform; the Bayes-optimal latent NLL over the gender atoms is $6 \log 2 \approx 4.16$ and the target is the calibrated posterior over the four modes (RS diagnostics after [4,24]).

Compiling both as one SDD is intractable (the two disjunctive subsystems multiply out, §6), so we compile two circuits over the same individuals – a 114-atom digit and a 69-atom gender circuit – driven by one shared CNN with image-free gender heads. The split only separates the two independent subsystems; the reasoning shortcut lives entirely within the jointly-compiled gender circuit, so the target posterior is unaffected.

Table 8 (App. H) summarizes the outcome. The CNN recovers the digits through the circuit on every method (accuracy 0.99). For the gender RS, no factorized method covers the four modes. **Independent** hedges (NLL 6.93) but leaks mass everywhere (TV 1.00); **single-WMC** (Semantic Loss [37], conditioned DeepProbLog [23]) and the **learned mixture** (JustWMC, the BEARS-ensemble class [24]) lower the NLL only marginally (6.13 / 6.69) and do not spread (TV 0.99): the mixture-WMC objective is minimized by one confident head, so gradient descent collapses onto a seed-dependent mode. Representability does not imply learnability [15]. **Justification-anchored JustWMC** closes the gap *constructively*: seeding the four heads from the circuit’s enumerated completions and learning only the selector (which converges to uniform) realizes the positive direction of Theorem 5 in practice: Bayes-optimal NLL 4.17, zero concept ECE, and mode-coverage TV 0.02 with no seed variance. The compiled circuit supplies the multimodal joint that gradient descent over a factorized hypothesis class does not find. An explicit ensemble mitigation is consistent with this boundary between hypothesis classes. BEARS [24], a $K=4$ deep ensemble, spreads mass across the modes, reaching mode-coverage TV 0.44 where the single-mode methods sit at 0.99–1.00. It stays far from the anchored posterior nonetheless (TV 0.44 against 0.02, ECE 0.41 against 0.00, NLL 6.36 against 4.17) and varies across seeds (± 1.1 TV, ± 2.36 NLL). A learned mixture therefore does not recover the multimodal joint that anchoring reads directly from the circuit.

4.3 Size comparison with DeepProbLog

DeepProbLog (DPL) [23] compiles a probabilistic logic program to an sd-DNNF, the same arithmetic-circuit class our system builds on Horn inputs. On the MNIST-*SRQIQ* ontology, **12 of 52 axioms (23%) leave ProbLog’s Horn fragment** and need hand-written constraint encodings; none is strictly inexpressible, but each abandons the Horn structure DPL compiles natively (Table 10, App. H).

5 Related work

The closest concurrent work is the SDD compiler of [20], which also trains a NeSy model through a circuit compiled from a DL ontology; we extend the recipe to *SR_{OTQ}*, drive compilation by a consequence-based calculus [32,33] rather than synthesizing the circuit from the semantics, and add a mechanized metatheory. That line addresses neither the reasoning shortcuts we characterize and mitigate (§3.2) nor real perception (it reports on hand-built feature vectors); [9,10] survey the broader OWL-aware NeSy landscape.

The DL embeddings of §1, the lattice-saturated embeddings of [41], and fuzzy-DL variants [35,40] optimize a relaxation of OWL semantics on a continuous representation; they scale but compute no classical WMC. Embed2Sym [2] clusters embeddings to recover symbols. Within knowledge-compilation NeSy, DeepProbLog [23] compiles to sd-DNNF; Semantic Loss [37] and Semantic Probabilistic Layers [1] train against a propositional WMC; A-NeSI [18] amortizes it; NeuPSL [27], Scallop [21], DeepStochLog [34], NeurASP [38], and NeSyDM [16] target propositional, Datalog, or answer-set programs; we extend the lineage to OWL 2 DL.

Reductions of expressive DLs to rule languages predate NeSy (KAON2: *SHIQ* to disjunctive Datalog [12]); a reduction alone yields no learner, but BAOBAB adds the path to an exact, differentiable WMC.

The independence assumption [25,24,15,17] is the limit we attack; our experiments give a *SR_{OTQ}* instance of the BEARS recipe with an RS-awareness characterization (Theorem 5). Other probabilistic-DL systems differ in target: DISPONTE [30] fixes the world-sum; BUNDLE [29] and TRILL [39] enumerate tableau explanations rather than one fixed circuit; LNNs [28] use user-set rule weights.

6 Limitations and future work

The guarded saturation that drives compilation (App. C) is sound but bounded, so it misses consequences of a deep ungrounded existential chain. Closing that gap calls for the full disjunctive context calculus [32,33], sound and complete under the trivial expansion strategy but not yet convergent on large ontologies; we treat it as a separate development. Our benchmarks sidestep the gap with grounding-covered, function-free constructs (Theorem 1). Multi-individual ABoxes inflate the ground theory by $O(|\Delta|^2)$ in role atoms and $O(|\Delta|^3)$ in the equality theory; larger ABoxes will need pairwise blocking, a sampling grounder, or amortized inference [18,16]. One of the three Pizzaiolo grounding refinements, the closed-world reading of the *NonVegetarianPizza* and *SpicyPizza* qualified existentials, is a genuine departure from the open-world Tarskian semantics of *SR_{OTQ}*: it materializes the biconditional $X(a) \Leftrightarrow \exists R.C$ so a latent class can be inferred from the absence of a filler, which the verified grounding (Theorem 3) does not license. It is opt-in and confined to declared families; App. D ablates it, and latent recovery degrades from 0.92 to 0.78 without it (the model becomes over-permissive rather than

failing), so the refinement trades open-world faithfulness for identifiability under partial supervision. Datatypes are out of scope. The DeepProbLog comparison (§4.3) counts the axioms whose encoding leaves ProbLog’s Horn fragment (23%, an expressivity count, not an accuracy claim); it does not run DeepProbLog end to end. Our Independent, single-WMC, and learned-mixture rows are controlled ablations of one pipeline: no external NeSy system compiles a non-Horn ontology for a head-to-head run. In sum, the method’s scope rests on five assumptions, each stated where used: a finite active domain; exactness only on grounding-covered or function-free existentials; bounded, sound-but-incomplete saturation; the opt-in Pizzaiolo grounding refinements; and, for anchored JustWMC, a #RS small enough to enumerate. Scaling the perception to many-individual scenes is the natural next step, as is evaluating anchored JustWMC beyond the enumerable-#RS regime, on a reasoning-shortcut benchmark whose intended concepts are identifiable from the perceptual input under limited concept supervision, against an external mitigation baseline.

7 Conclusion

SRQIQ admits neuro-symbolic learning by knowledge compilation: consequence-based saturation over *ALCHOQ* plus ABox grounding of the *SRQIQ* features produce an SDD whose evidence-conditioned WMC trains perception under partial supervision. Where supervision underdetermines the latents, anchoring the mixture to the circuit’s *enumerated justifications* recovers the Bayes-optimal posterior that learned mixtures miss [17].

References

1. Ahmed, K., Teso, S., Van den Broeck, G., Chang, K.W., Vergari, A.: Semantic probabilistic layers for neuro-symbolic learning. In: Advances in Neural Information Processing Systems (2022)
2. Aspis, Y., Broda, K., Lobo, J., Russo, A.: Embed2Sym: Scalable neuro-symbolic reasoning via clustered embeddings. In: Principles of Knowledge Representation and Reasoning (2022)
3. Bate, A., Motik, B., Cuenca Grau, B., Simăncik, F., Horrocks, I.: Extending consequence-based reasoning to SRIQ. In: Principles of Knowledge Representation and Reasoning (2016)
4. Bortolotti, S., Marconato, E., Carraro, T., Morettin, P., van Krieken, E., Vergari, A., Teso, S., Passerini, A.: A neuro-symbolic benchmark suite for concept quality and reasoning shortcuts. In: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track (2024), arXiv:2406.10368
5. Bourguin, G., Lewandowski, A.: Pizzaiolo dataset: ontologically explainable synthetic pizza images (2023). <https://doi.org/10.5281/zenodo.10165941>, sysReIC, LISIC, Université du Littoral Côte d’Opale; CC BY-NC 4.0
6. Chen, J., Hu, P., Jiménez-Ruiz, E., Holter, O.M., Antonyrajah, D., Horrocks, I.: OWL2Vec*: Embedding of OWL ontologies. Machine Learning **110**(7), 1813–1845 (2021)

7. Darwiche, A.: SDD: A new canonical representation of propositional knowledge bases. In: International Joint Conference on Artificial Intelligence (2011)
8. Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: International Conference on Machine Learning (2017)
9. Herron, D., Jiménez-Ruiz, E., Weyde, T.: On the benefits of OWL-based knowledge graphs for neural-symbolic systems. In: Neural-Symbolic Learning and Reasoning. CEUR Workshop Proceedings, vol. 3432, pp. 327–335 (2023)
10. Herron, D., Jiménez-Ruiz, E., Weyde, T.: On the potential of logic and reasoning in neurosymbolic systems using OWL-based knowledge graphs. *Neurosymbolic Artificial Intelligence* (2025)
11. Horrocks, I., Kutz, O., Sattler, U.: The even more irresistible SROIQ. In: Principles of Knowledge Representation and Reasoning (2006)
12. Hustadt, U., Motik, B., Sattler, U.: Reasoning in description logics by a reduction to disjunctive datalog. *Journal of Automated Reasoning* **39**(3), 351–384 (2007). <https://doi.org/10.1007/s10817-007-9080-3>
13. Jackermeier, M., Chen, J., Horrocks, I.: Dual box embeddings for the description logic EL^{++} . In: Web Conference (2024)
14. Kazakov, Y., Krötzsch, M., Simánčík, F.: The incredible ELK: From polynomial procedures to efficient reasoning with EL ontologies. *Journal of Automated Reasoning* **53**, 1–61 (2014)
15. van Krieken, E., Minervini, P., Ponti, E.M., Vergari, A.: On the independence assumption in neurosymbolic learning. In: International Conference on Machine Learning (2024), arXiv:2404.08458
16. van Krieken, E., Minervini, P., Ponti, E.M., Vergari, A.: Neurosymbolic diffusion models. In: Advances in Neural Information Processing Systems (2025)
17. van Krieken, E., Minervini, P., Ponti, E.M., Vergari, A.: Neurosymbolic reasoning shortcuts under the independence assumption. In: Neurosymbolic Learning and Reasoning (2025)
18. van Krieken, E., Thanapalasingam, T., Tomczak, J.M., van Harmelen, F., ten Teije, A.: A-NeSI: A scalable approximate method for probabilistic neurosymbolic inference. In: Advances in Neural Information Processing Systems (2023)
19. Kulmanov, M., Liu-Wei, W., Yan, Y., Hoehndorf, R.: EL embeddings: Geometric construction of models for the description logic EL^{++} . In: International Joint Conference on Artificial Intelligence (2019)
20. Lazzari, N., Presutti, V., Vergari, A.: To neuro-symbolic classification and beyond by compiling description logic ontologies to probabilistic circuits. arXiv preprint arXiv:2601.14894 (2026)
21. Li, Z., Huang, J., Naik, M.: Scallop: A language for neurosymbolic programming. In: Programming Language Design and Implementation (2023)
22. Lord, P., Gehrke, B., De Bortoli, F., Larralde, M.: **horned-owl**: Building ontologies at big data scale. Rust library, <https://github.com/phillord/horned-owl> (2023)
23. Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., De Raedt, L.: Deep-ProbLog: Neural probabilistic logic programming. In: Advances in Neural Information Processing Systems (2018)
24. Marconato, E., Bortolotti, S., van Krieken, E., Vergari, A., Passerini, A., Teso, S.: BEARS make neuro-symbolic models aware of their reasoning shortcuts. In: Uncertainty in Artificial Intelligence (2024)
25. Marconato, E., Teso, S., Vergari, A., Passerini, A.: Not all neuro-symbolic concepts are created equal: Analysis and mitigation of reasoning shortcuts. In: Advances in Neural Information Processing Systems (2023)

26. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Conference on Automated Deduction (2021)
27. Pryor, C., Dickens, C., Augustine, E., Albalak, A., Wang, W.Y., Getoor, L.: NeuPSL: Neural probabilistic soft logic. In: International Joint Conference on Artificial Intelligence (2023)
28. Riegel, R., Gray, A., Luus, F., Khan, N., Makondo, N., Akhalwaya, I.Y., Qian, H., Fagin, R., et al.: Logical neural networks. arXiv preprint arXiv:2006.13155 (2020)
29. Riguzzi, F., Bellodi, E., Lamma, E., Zese, R.: BUNDLE: A reasoner for probabilistic ontologies. In: Web Reasoning and Rule Systems (2013)
30. Riguzzi, F., Bellodi, E., Lamma, E., Zese, R.: Probabilistic description logics under the distribution semantics. *Semantic Web* **6**(5), 477–501 (2015)
31. Tang, Z., Hinnerichs, T., Peng, X., Zhang, X., Hoehndorf, R.: FALCON: Faithful neural semantic entailment over ALC ontologies. arXiv preprint arXiv:2208.07628 (2022)
32. Tena Cucala, D., Cuenca Grau, B., Horrocks, I.: Consequence-based reasoning for description logics with disjunction, inverse roles, number restrictions, and nominals. In: International Joint Conference on Artificial Intelligence (2018)
33. Tena Cucala, D., Cuenca Grau, B., Horrocks, I.: Pay-as-you-go consequence-based reasoning for the description logic SROIQ. *Artificial Intelligence* **298**, 103518 (2021)
34. Winters, T., Marra, G., Manhaeve, R., Raedt, L.D.: DeepStochLog: Neural stochastic logic programming. In: AAAI Conference on Artificial Intelligence (2022)
35. Wu, X., Zhu, X., Zhao, Y., Dai, X.: Differentiable fuzzy ALC: A neural-symbolic representation language for symbol grounding. arXiv preprint arXiv:2211.12006 (2022)
36. Xiong, B., Potyka, N., Tran, T.K., Nayyeri, M., Staab, S.: Faithful embeddings for EL^{++} knowledge bases. In: International Semantic Web Conference (2022)
37. Xu, J., Zhang, Z., Friedman, T., Liang, Y., Van den Broeck, G.: A semantic loss function for deep learning with symbolic knowledge. In: International Conference on Machine Learning (2018)
38. Yang, Z., Ishay, A., Lee, J.: NeurASP: Embracing neural networks into answer set programming. In: International Joint Conference on Artificial Intelligence (2020)
39. Zese, R., Bellodi, E., Lamma, E., Riguzzi, F., Cota, G.: Tableau reasoning for description logics and its extension to probabilities. *Annals of Mathematics and Artificial Intelligence* **82**(1–3), 101–130 (2018)
40. Zhao, Y.: Fast and faithful: Scalable neuro-symbolic learning and reasoning with differentiable fuzzy EL^{++} . In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 1987–1997 (2026)
41. Zhapa-Camacho, F., Hoehndorf, R.: Lattice-based ALC ontology embeddings with saturation. *Neurosymbolic Artificial Intelligence* (2025)

A *SROIQ* syntax and semantics

The body (§2) summarizes the fragment; we give the full definition here for completeness. A *SROIQ* knowledge base is a triple $(\mathcal{T}, \mathcal{R}, \mathcal{A})$ over a signature of atomic concept names N_C , atomic role names N_R , and individual names N_I .

A *role* is an atomic role $r \in N_R$, an inverse role r^- , or the universal role U . *Concepts* are generated by

$$C, D ::= A \mid \top \mid \perp \mid \{a\} \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists R.C \mid \forall R.C \mid \geq n R.C \mid \leq n R.C \mid \exists R.\text{Self},$$

where $A \in \mathbf{N}_C$, $a \in \mathbf{N}_I$, R a role, and $n \in \mathbb{N}$. In the implementation (`baobab/sroi/syntax.py`) these are the dataclasses `ConceptName`, `Top`, `Bottom`, `Nominal`, `Not`, `And`, `Or`, `Exists`, `Forall`, `AtLeast`, `AtMost`, `HasSelf`; roles are `RoleName`, `InverseRole`, `UniversalRole`. `And/Or` hold a *flattened, deduplicated* frozen set of operands.

Axioms are organized into three boxes. The *TBox* $\mathcal{T}$ is a set of general concept inclusions $C \sqsubseteq D$ (with $C \equiv D$ abbreviating the two inclusions and $\text{Disjoint}(C, D)$ abbreviating $C \sqcap D \sqsubseteq \perp$). The *ABox* $\mathcal{A}$ is a set of concept assertions $C(a)$, role assertions $R(a, b)$, and (in)equalities $a \approx b$, $a \not\approx b$. The *RBox* $\mathcal{R}$ contains role inclusions $R \sqsubseteq S$, role chains $R_1 \circ \dots \circ R_k \sqsubseteq S$, and the role characteristics $\text{Trans}(R)$, $\text{Sym}(R)$, $\text{Asym}(R)$, $\text{Refl}(R)$, $\text{Irrefl}(R)$, $\text{Func}(R)$, $\text{InvFunc}(R)$, $\text{Disj}(R, S)$. (As usual for decidability, *SRQ* requires the RBox to be regular and number restrictions to use only simple roles; our benchmarks satisfy both.)

The semantics is Tarskian. An interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ has a non-empty domain $\Delta^{\mathcal{I}}$ and maps each $A \in \mathbf{N}_C$ to $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$, each $r \in \mathbf{N}_R$ to $r^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, and each $a \in \mathbf{N}_I$ to $a^{\mathcal{I}} \in \Delta^{\mathcal{I}}$. Roles extend by $(r^-)^{\mathcal{I}} = \{(y, x) : (x, y) \in r^{\mathcal{I}}\}$ and $U^{\mathcal{I}} = \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, and concepts by

$$\begin{aligned} \top^{\mathcal{I}} &= \Delta^{\mathcal{I}}, & \perp^{\mathcal{I}} &= \emptyset, & \{a\}^{\mathcal{I}} &= \{a^{\mathcal{I}}\}, \\ (\neg C)^{\mathcal{I}} &= \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}, & (C \sqcap D)^{\mathcal{I}} &= C^{\mathcal{I}} \cap D^{\mathcal{I}}, & (C \sqcup D)^{\mathcal{I}} &= C^{\mathcal{I}} \cup D^{\mathcal{I}}. \end{aligned}$$

$$\begin{aligned} (\exists R.C)^{\mathcal{I}} &= \{x : \exists y. (x, y) \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}, \\ (\forall R.C)^{\mathcal{I}} &= \{x : \forall y. (x, y) \in R^{\mathcal{I}} \rightarrow y \in C^{\mathcal{I}}\}, \\ (\geq n R.C)^{\mathcal{I}} &= \{x : \#\{y : (x, y) \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\} \geq n\}, \\ (\exists R.\text{Self})^{\mathcal{I}} &= \{x : (x, x) \in R^{\mathcal{I}}\}, \end{aligned}$$

with $\leq n R.C$ dual to $\geq (n+1) R.C$ under negation. $\mathcal{I}$ satisfies $C \sqsubseteq D$ iff $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$, $R \sqsubseteq S$ iff $R^{\mathcal{I}} \subseteq S^{\mathcal{I}}$, $R_1 \circ \dots \circ R_k \sqsubseteq S$ iff $R_1^{\mathcal{I}} \circ \dots \circ R_k^{\mathcal{I}} \subseteq S^{\mathcal{I}}$, and the role characteristics under their standard relational readings (Trans : transitive; $\text{Func}(R)$: $R^{\mathcal{I}}$ is right-unique; $\text{Disj}(R, S)$: $R^{\mathcal{I}} \cap S^{\mathcal{I}} = \emptyset$; etc.). This is the semantics our Lean development takes as the definition of truth (§J).

B Normalization and clausification

The compiler (`baobab/sroi/normalisation.py`) maps each axiom to *DL-clauses* $\bigwedge_i B_i \rightarrow \bigvee_j H_j$ over the term language of variables x, y , individual constants a , auxiliary constants o_ρ (introduced at grounding time for nominal / cardinality witnesses), and unary Skolem terms $f(t)$. Atoms are concept atoms $C(t)$, role atoms $R(t_1, t_2)$, and equalities $t_1 \approx t_2$ (`dlclauses.py`).

The first step is negation-normal form. `nnf` pushes negation to atomic concepts and nominals using the de Morgan, quantifier, and cardinality dualities:

$$\begin{aligned} \neg \top &\rightsquigarrow \perp, & \neg \perp &\rightsquigarrow \top, & \neg \neg C &\rightsquigarrow C, \\ \neg (C \sqcap D) &\rightsquigarrow \neg C \sqcup \neg D, & \neg (C \sqcup D) &\rightsquigarrow \neg C \sqcap \neg D, & \neg \exists R.C &\rightsquigarrow \forall R. \neg C, \\ \neg \forall R.C &\rightsquigarrow \exists R. \neg C, & \neg (\geq n R.C) &\rightsquigarrow \leq (n-1) R.C, & \neg (\leq n R.C) &\rightsquigarrow \geq (n+1) R.C, \\ \neg (\geq 0 R.C) &\rightsquigarrow \perp, \end{aligned}$$

$\neg A$ and $\neg\{a\}$ remain (atomic literals); $\neg\exists R.\text{Self}$ is preserved for grounding.

The structural transformation follows. Each non-atomic subconcept D is assigned a fresh concept name Q_D (method `q`) and defined by clauses for $Q_D \equiv D$ (method `_define`); a *SROIQ* inclusion $C \sqsubseteq D$ then becomes the single clause $Q_C(x) \rightarrow Q_D(x)$. The definitional clauses are:

- $Q \equiv C \sqcap D$: $Q(x) \rightarrow Q_C(x)$, $Q(x) \rightarrow Q_D(x)$, $Q_C(x) \wedge Q_D(x) \rightarrow Q(x)$.
- $Q \equiv C \sqcup D$: $Q(x) \rightarrow Q_C(x) \vee Q_D(x)$, $Q_C(x) \rightarrow Q(x)$, $Q_D(x) \rightarrow Q(x)$.
- $Q \equiv \exists R.C$: $Q(x) \rightarrow R(x, f_Q(x))$, $Q(x) \rightarrow Q_C(f_Q(x))$, $R(x, y) \wedge Q_C(y) \rightarrow Q(x)$, where f_Q is a unary Skolem function.
- $Q \equiv \forall R.C$: $Q(x) \wedge R(x, y) \rightarrow Q_C(y)$.
- $Q \equiv \neg A$: $Q(x) \wedge A(x) \rightarrow \perp$, $\rightarrow Q(x) \vee A(x)$.
- $Q \equiv \exists R.\text{Self}$: $Q(x) \rightarrow R(x, x)$, $R(x, x) \rightarrow Q(x)$.
- $Q \equiv \geq n R.C$: for $i = 0, \dots, n-1$, $Q(x) \rightarrow R(x, f_{Q,i}(x))$ and $Q(x) \rightarrow Q_C(f_{Q,i}(x))$; pairwise distinctness $Q(x) \wedge f_{Q,i}(x) \approx f_{Q,j}(x) \rightarrow \perp$ for $i < j$; closure $R(x, y) \wedge Q_C(y) \rightarrow Q(x)$ when $n = 1$.
- $Q \equiv \leq n R.C$: $Q(x) \wedge \bigwedge_{t=0}^n (R(x, y_t) \wedge Q_C(y_t)) \rightarrow \bigvee_{i < j} y_i \approx y_j$ ($n+1$ fresh successor variables).

Nominals $\{a\}$ are not given definitional clauses: they map to a stable proxy Nom_a and an entry $\text{Nom}_a \mapsto a$ in the `GroundHooks` record. Qualified number restrictions additionally register a tuple (Q, n, R, Q_C) in `hooks.at_least` / `hooks.at_most`; inverse roles register (R, S) in `hooks.role_inverses` and resolve to a proxy role name; the universal role emits the fact $\rightarrow U(x, y)$. RBox characteristics are routed either to native clauses (symmetric: $R(x, y) \rightarrow R(y, x)$; reflexive: $\rightarrow R(x, x)$; irreflexive: $R(x, x) \rightarrow \perp$; functional: $R(x, y_0) \wedge R(x, y_1) \rightarrow y_0 \approx y_1$; inverse-functional dually) or to the grounding record (transitive, chains, asymmetric, disjoint, nominal).

C Consequence-based saturation

The DL-clause set is saturated under propositional *hyperresolution with most-general-unifier matching* (`baobab/sroi/cb_saturation.py`): from $\Gamma \rightarrow \Delta \vee A$ and $A' \wedge \Gamma' \rightarrow \Delta'$ with $A\sigma = A'\sigma$ the MGU resolvent $(\Gamma \wedge \Gamma')\sigma \rightarrow (\Delta \vee \Delta')\sigma$ is added, unification using Robinson’s algorithm with occurs-check over the four term kinds. A clause is admitted to the pool only if it passes four guards that bound the otherwise unbounded context machinery and force termination:

1. **tautology elimination**: drop any clause whose head and body share an atom;
2. **variable bound**: at most $\max(2, \text{max-vars-in-input})$ distinct variables per clause (two suffices for *ALCHQ*; transitivity and two-step chains need three; $\leq n$ needs $n+2$);
3. **role-atom bound**: at most $\max(2, \text{max-role-atoms-in-input})$ role atoms per body (a length- k chain needs k ; $\leq n$ needs $n+1$);

4. **term-depth bound:** every function term has nesting depth ≤ 1 , blocking unbounded Skolem chains $f(g(\dots))$.

The fixpoint loop re-resolves all clause pairs until a full pass adds nothing.

Lemma 1 (Termination and soundness of the guarded saturation). *The guarded saturation terminates, and every clause it derives is a logical consequence of its input.*

Proof. Termination: the four guards keep the reachable atom-and-clause universe finite (bounded variables, bounded role atoms, depth- ≤ 1 terms over a finite signature) and tautologies are removed, so the subsumption-minimal pool is finite and the fixpoint loop halts. Soundness: each resolution step adds only entailed clauses; the property is mechanized for the underlying $\mathcal{ALCHOQ}$ saturation (axiom-free; §J).

The calculus forgoes *completeness*: the bounded calculus need not derive every subsumption that the full $\mathcal{ALCHOIQ}$ context calculus [32,33] would. Incompleteness is harmless for the consequences a ground-time grounding already discharges (nominals, number restrictions, local reflexivity, role characteristics): there the grounded CNF Θ_Δ is faithful on its own (§D), so any clause saturation adds is already entailed by Θ_Δ and changes neither the model set nor, over a fixed vtree, the circuit (§F). It is *not* harmless for existential restrictions grounding does not cover: their consequences enter Θ_Δ only through saturation, so an incomplete or skipped saturation yields a sound but over-permissive circuit that admits models the ontology forbids. The trade-off is deliberate: we bound the calculus for termination and accept incompleteness on deep existential chains. On a grounding-covered, function-free fragment saturation adds nothing (Theorem 1), so we skip it and ground directly (§6).

D Grounding the $\mathcal{SROIQ}$ features

At grounding time the **GroundHooks** record is materialized over the active domain Δ ($d = |\Delta|$; `baobab/grounding/`). Each grounding rule emits a bounded number of propositional clauses, and each is justified by a Tarskian-semantics soundness lemma (§J). Table 2 lists every **GroundHooks** rule with its emitted clause schema and ground-clause count; the three optional refinements of §4.1 are documented separately below; the closed qualified-existential refinement is ablated in Table 3.

The grounder also allocates a propositional atom for every concept name applied to each individual ($O(|N_C| d)$) and every role applied to each ordered pair ($O(|N_R| d^2)$); when the ontology uses nominals, qualified number restrictions, or (inverse-)functional roles it additionally emits an equality theory (d^2 atoms with reflexivity, symmetry, transitivity, and concept/role congruence), otherwise the equality predicate is omitted (refinement (ii) below). The dominant terms are the cubic transitivity / equality-transitivity clauses and, for a length- k chain or a number restriction of bound n , the $d^{k+1} / O(d^{n+1})$ entries, the polynomial-degree bounds proved in our Lean development (§J).

Table 2. Grounding schemata over the active domain Δ , $d = |\Delta|$. a, b, c range over Δ ; $\binom{d}{n}$ is the n -subset count.

feature	emitted ground clauses	count
nominal Nom_a	$\text{Nom}_a(a); \neg \text{Nom}_a(b), b \neq a$	d
$\geq n R.C$ at x	$Q(a) \leftrightarrow \bigvee_{ S =n} \bigwedge_{y \in S} R(a, y) \wedge C(y)$	$d \left(\binom{d}{n} + 1 \right)$
$\leq n R.C$ at x	$Q(a) \leftrightarrow \bigwedge_{ T =n+1} \left(\bigwedge_{y \in T} R(a, y) \wedge C(y) \rightarrow \bigvee_{i < j} y_i \approx y_j \right)$	$O(d^{n+1})$
inverse $R \equiv S^-$	$R(a, b) \leftrightarrow S(b, a)$	$2d^2$
$\text{Trans}(R)$	$R(a, b) \wedge R(b, c) \rightarrow R(a, c)$	$d^3 - d$
chain $R_1 \circ \dots \circ R_k \sqsubseteq S$	$R_1(a_0, a_1) \wedge \dots \wedge R_k(a_{k-1}, a_k) \rightarrow S(a_0, a_k)$	d^{k+1}
$\text{Sym}(R)$	$R(a, b) \rightarrow R(b, a)$	$d^2 - d$
$\text{Asym}(R)$	$\neg (R(a, b) \wedge R(b, a)), a \neq b$	$d^2 - d$
$\text{Disj}(R, S)$	$\neg (R(a, b) \wedge S(a, b))$	d^2
$\text{Refl}(R)$	$R(a, a)$	d
$\text{Irrefl}(R)$	$\neg R(a, a)$	d
$\text{Func}(R)$	$R(a, b) \wedge R(a, c) \rightarrow b \approx c$	$d^2 - d$
$\text{InvFunc}(R)$	$R(b, a) \wedge R(c, a) \rightarrow b \approx c$	$d^2 - d$
$\exists R.\text{Self proxy } Q$	$Q(a) \leftrightarrow R(a, a)$	$2d$
universal U	$U(a, b)$	d^2

Three optional refinements. The role-typing, equality-skip, and closed-existential refinements used for the Pizzaiolo fragment (§4.1) are opt-in and off by default, so every other circuit in the paper is byte-identical to the unrefined grounding. Refinements (i) and (ii) are both instances of one fact about weighted model counting: it factorizes over an independent partition of the atoms, and a block whose sub-theory has a unique satisfying assignment contributes a single constant factor. We mechanize that fact and the two refinements as model-count-preservation lemmas (`wmc_split`, `wmc_factor`, `wmc_prune_forcedFalse`, `wmc_skip_equality`, with exact count preservation `wmc_card_preserved` and constant log-offset `wmc_log_offset`; §J). Refinement (iii) is a closed-world reading justified only by the Tarskian argument below. (i) *Domain/range typing*. When a role R declares a domain D_R and range E_R and the named individuals are typed accordingly, $R(a, b)$ is false in every model unless $a \in D_R$ and $b \in E_R$, so allocating its atom only for those pairs replaces the $|\mathbf{N}_R| d^2$ role atoms by $\sum_R |D_R| |E_R|$. The pruned atoms form an independent block whose only model sets them all false, so the model count is preserved exactly and the WMC up to the constant $\prod w(\text{false})$. (ii) *Equality skip*. On an ontology free of nominals, number restrictions, and functional roles, no clause outside the equality theory mentions $\approx$; the $\approx$ atoms then form an independent sub-theory with, under the unique-name default, a single model, so dropping it preserves the model count exactly and the weighted count up to a constant (the training gradient and the consistency check are identical). (iii) *Closed existential*. When a role's fillers are closed to the named domain, a definition $X \equiv \exists R.C$ whose head Skolem witness the grounder drops (§3.1) is recovered as the exact biconditional $X(a) \leftrightarrow \bigvee_{b \in \Delta} R(a, b) \wedge C(b)$ ($2d$ clauses), restoring the backward and closure directions ($X \rightarrow \exists$ and $\neg \exists \rightarrow \neg X$) that Skolemization omits. Unlike (i) and (ii)

Table 3. Closed-world ablation on the Pizzaiolo fragment (mean $\pm$ std over ten seeds, five epochs; same fragment and metrics as Table 7). *Closed* materializes the exact biconditional $X(a) \Leftrightarrow \bigsqcup_t \text{hasTopping}(a, t) \sqcap C(t)$; *open* keeps the forward direction $X(a) \rightarrow \exists \text{hasTopping}.C$ only, dropping the closure clause $\exists \text{hasTopping}.C \rightarrow X(a)$. Removing the closure clause leaves supervised toppings untouched but lowers latent property recovery from 0.92 to 0.78; the reported violation falls to 0.00 because the removed clause is the constraint whose violation was counted (§4.1).

refinement method		topping property		viol $\downarrow$
closed	Independent	0.95 $\pm$.01	0.45 $\pm$.15	0.89 $\pm$.18
closed	WMC	0.93 $\pm$.01	0.92$\pm$.02	0.06 $\pm$.02
open	Independent	0.95 $\pm$.01	0.45 $\pm$.16	0.86 $\pm$.28
open	WMC	0.95 $\pm$.01	0.78 $\pm$.01	0.00 $\pm$.00

Algorithm 1: Compile a *SRQIQ* ontology to a differentiable circuit and train the perception network

Input: ontology $\mathcal{O}$, active domain Δ , perception network p_θ , supervised instances $\{(x^{(k)}, e^{(k)})\}$

```

/* Compile (once): */
1  $N \leftarrow \text{NNF}(\mathcal{O});$  // normalization, App. B
2  $\mathcal{C} \leftarrow \text{Clausify}(N);$  // structural transformation
3  $\mathcal{C}^* \leftarrow \text{Saturate}(\mathcal{C});$  // hyperresolution; skip when grounding-covered
4  $\Theta_\Delta \leftarrow \text{Ground}(\mathcal{C}^*, \Delta);$  // expand nominals, number restrictions, roles
5  $S \leftarrow \text{CompileSDD}(\Theta_\Delta);$  // PySDD over a fixed vtree
/* Train (per step): */
6 foreach instance  $(x^{(k)}, e^{(k)})$  do
7    $p \leftarrow p_\theta(x^{(k)});$  // per-atom probabilities
8    $\ell \leftarrow -\log \text{WMC}(S, p, e^{(k)});$  // evidence-conditioned
9   backpropagate  $\ell$  through  $S$  into  $\theta$ ;

```

this is a closed-world reading: it is sound only under the named-filler closure assumption, which holds for the Pizzaiolo dataset (each pizza’s topping list is fully observed) but is not an open-world consequence and is therefore outside the mechanized guarantees.

E SDD compilation, WMC, and the distribution semantics

Algorithm 1 states the end-to-end workflow referenced in §3.1: the five compile-time stages run once per ontology, and the differentiable WMC pass runs once per training step.

The grounded CNF is compiled to an SDD with PySDD. The differentiable WMC layer (`baobab/nesy/wmc_layer.py`, class `WmcLayer`) evaluates the circuit bottom-up: a positive literal for atom a contributes its probability p_a , a negative literal $1 - p_a$, the `true/false` constants contribute 1/0, and a decision node with prime/sub partition $\{(p_i, s_i)\}$ contributes $\sum_i \text{WMC}(p_i) \cdot \text{WMC}(s_i)$, memoized so

each node is visited once. Probabilities are clamped to $[\varepsilon, 1-\varepsilon]$ for log-stability; the pass is exact in floating point and differentiable by autodiff, so gradients flow into p_θ . Evidence conditioning (`wmc_with_evidence`) fixes $p_a = 1$ (resp. 0) for each $(a, \text{true}) \in e$ (resp. `false`) and counts on the resulting weights, yielding

$$\text{WMC}(\Theta \wedge e \mid p_\theta) = \sum_{\substack{\alpha \models \Theta \\ \alpha \supseteq e}} \prod_a p_a^{\alpha_a} (1 - p_a)^{1 - \alpha_a},$$

the sum over satisfying assignments α of their product weight (Theorem 4 states this in full).

Proof (Proof of Theorem 4). Reading each atom as an independent Bernoulli event with parameter p_a is exactly the distribution semantics [30]: the displayed expression is the total probability mass of the interpretations that satisfy $\Theta \wedge e$, the world-sum, and taking $e = \emptyset$ gives the unconditioned statement of Theorem 4. The bottom-up evaluation above computes exactly this sum, and the correspondence on the compiled circuit is mechanized in Lean (§J).

The world-sum is the quantity the losses of §G optimize.

F Saturation and circuit size

§3.1 divides a construct’s consequences into those a grounding discharges and those only saturation can supply. Here we make that division precise on two axes: circuit *size*, on which saturation is provably inert, and the *model set*, where saturation is decisive for existentials that no grounding rule covers. Recall that the compiler (§E) builds the SDD over a *fixed* vtree, conjoining the ground clauses with PySDD’s `Apply`, which keeps every node compressed and trimmed.

Theorem 1 (Saturation is circuit-invariant under a fixed vtree). *Fix a vtree v . Let Θ_Δ be the grounded CNF and let Θ_Δ^+ add to it any set of clauses entailed by Θ_Δ (as sound saturation does, Lemma 1). The compressed, trimmed SDDs of Θ_Δ and Θ_Δ^+ over v are identical; in particular they have the same node count and the same weighted model count.*

Proof. Saturation resolves over the existing signature and introduces no new ground atoms, so Θ_Δ^+ is a CNF over the same variables as Θ_Δ and is compiled over the same vtree. Sound saturation adds only entailed clauses (Lemma 1), so $\Theta_\Delta^+ \equiv \Theta_\Delta$ as Boolean functions. For a fixed vtree the compressed and trimmed SDD of a function is unique [7], and `Apply` maintains compression and trimming, so the compiler returns this canonical diagram regardless of which equivalent clause set presents the function. The two SDDs, and hence their sizes and weighted model counts, therefore coincide. (The fixed vtree is essential: over a different variable order the same function can compile to a different size, so the claim is about adding clauses, not about reordering.)

Table 4. Effect of saturation on SDD node count, measured with the accompanying library. Saturation adds entailed clauses (column *clauses*) without changing the atom set. Over a *fixed* vtree the size is identical with and without saturation (Theorem 1); under dynamic vtree *minimization* it usually shrinks but is not guaranteed to (**bird-penguin**).

ontology	clauses	fixed vtree	minimized	
	unsat→sat	unsat = sat	unsat	sat
horn-chain	$3 \rightarrow 6$	10	8	8
disjunction	$7 \rightarrow 12$	23	17	12
two-disjunctions	$11 \rightarrow 29$	38	22	18
exists+disjunction	$8 \rightarrow 73$	73	41	36
bird-penguin	$11 \rightarrow 44$	1011	139	143

Consequently, under the fixed-vtree compilation we use, saturation is *exactly* size-neutral: it cannot shrink the circuit (nor enlarge it). A size reduction becomes available only once the compiler is allowed to search for a vtree, where the compiled size is no longer canonical across logically equivalent inputs.

Proposition 1 (Under vtree minimization the reduction is real but not monotone). *With dynamic vtree minimization enabled, there are ontologies on which saturation strictly decreases the minimized SDD size and ontologies on which it strictly increases it.*

Proof. By the witnesses in Table 4: for **disjunction** the minimized size drops from 17 to 12 after saturation, while for **bird-penguin** it rises from 139 to 143.

Every row of Table 4 is an ontology on which grounding is already faithful (the constructs are function-free or grounding-covered), so saturation only re-derives clauses already entailed by Θ_Δ and Theorem 1 applies. The other regime is where saturation is *not* redundant.

Proposition 2 (Saturation is load-bearing for ungrounded existentials). *There are ontologies on which skipping saturation strictly enlarges the model set and drops an entailment that holds in every model of the ontology.*

Proof. Take $A \sqsubseteq \exists R.B$, $B \sqsubseteq C$, $\exists R.C \sqsubseteq D$ with $A(a)$ over $\Delta = \{a\}$; the ontology entails $D(a)$. With saturation the grounded theory has 6 models and forces $D(a)$; without it the head existential $A \sqsubseteq \exists R.B$ clausifies to Skolem-term clauses the grounder drops, the theory has 14 models, and $D(a)$ holds in only a 0.71 fraction of them. The two circuits compute different functions.

The two facts delimit the role of entailment exactly. Where grounding is already faithful, saturation cannot change the circuit (Theorem 1); where it is not, saturation is the only route by which an ungrounded existential’s consequences reach the finite theory (Proposition 2). Saturation’s job is *completeness*, not compression. Our benchmarks lie entirely in the first regime: every construct they use is function-free or grounding-covered, so their grounded circuits are faithful

Table 5. Compiling the *full* Pizzaiolo *SROIQ* ontology (66 named classes, 90 existential and 44 universal restrictions, right-hand-side disjunctions on *Pizza* and *SpicyPizza*) over a growing active domain, with a fixed vtree throughout. Grounding stays sub-second at every size; the compiled SDD grows super-linearly and exceeds a 22 GB / 600 s budget at three toppings. The 59-atom two-topping fragment of §4.1 is a separately extracted role-based sub-signature, not a row here.

toppings ($ \Delta $)	atoms	clauses	ground (s)	SDD nodes	peak mem
1 ($ \Delta =2$)	201	638	0.04	3,182	0.7 GB
2 ($ \Delta =3$)	302	957	0.19	12,064	5.7 GB
3 ($ \Delta =4$)	403	1,276	0.01	—	> 22 GB [†]

[†] compilation exceeds the 600 s / 22 GB budget; grounding still completes in 0.01 s.

(Theorem 3) and saturation is provably unnecessary for them; we therefore report results without relying on it, and without any wall-clock cutoff.

The full workflow of this section, normalization, consequence-based saturation, grounding, and SDD compilation, is implemented as an installable Python package (`baobab/sroiq/` in the released code), with the example ontologies used above under `experiments/ontologies/`; the measurements in Table 4 were produced with it.

Diagnosing the Pizzaiolo compilation wall. The full ontology of §4.1 is the object measured in Table 5; the 59-atom fragment we train on is a deliberately minimized role-based sub-signature, so its size is not comparable to the rows of that table. The three candidate bottlenecks separate. *Grounding size* is not the cause: the ground theory grows linearly (201 $\rightarrow$ 302 $\rightarrow$ 403 atoms) and grounding completes in under 0.2 s at every size, including the three-topping instance that fails to compile (0.01 s). The *vtree* is not the cause either: compilation uses a single fixed vtree throughout (Theorem 1), so the growth is a property of the compiled function, not of the variable order. The bottleneck is the *intrinsic circuit size*: the SDD grows super-linearly in the grounding (3,182 $\rightarrow$ 12,064 nodes, a $3.8\times$ jump for a $1.5\times$ clause increase), compile time rises $66\times$ (0.9 $\rightarrow$ 60 s) for one added topping, and peak memory exceeds 22 GB at three toppings. The mechanism is treewidth: the 67 disjointness axioms together with the right-hand-side disjunctions ($\text{Pizza} \sqsubseteq \text{VegetarianPizza} \sqcup \text{NonVegetarianPizza}$, $\text{SpicyPizza} \equiv \bigsqcup_i \text{Topping}_i$) and the *VegetarianPizza* complement couple the topping atoms across each pizza, raising the primal-graph treewidth, and SDD size is worst-case exponential in treewidth (§6). Grounding size feeds this growth but is not itself the wall, which is why a hand-pruned sub-signature compiles where the full ontology cannot.

G Learning objectives

Let $p_\theta(\cdot \mid x)$ be the perception’s per-atom posterior on example x and e its revealed evidence (`baobab/nesy/wmc_layer.py` and the `experiments/` drivers).

Independent. $\mathcal{L} = \frac{1}{|e|} \sum_{(a,v) \in e} \text{BCE}(p_\theta(a), v)$, binary cross-entropy on the observed atoms only; the constraint is ignored.

WMC. $\mathcal{L} = \text{BCE}(p_\theta, v)|_e + \lambda(-\log \text{WMC}(\Theta \wedge e \mid p_\theta))$ (Eq. 2); the second term is the Semantic-Loss/DeepProbLog objective on the compiled circuit.

JustWMC. A single `MixtureEncoder` with a shared MLP body, K atom heads μ_k , and a softmax selector π , trained on

$$\mathcal{L}_{\text{Just}} = \underbrace{\sum_k \pi_k \text{BCE}(\mu_k, v)|_e}_{\text{mixture label term}} + \lambda \left(-\log \sum_k \pi_k \text{WMC}(\Theta \wedge e \mid \mu_k) \right) - \kappa \cdot \frac{1}{K} \sum_k \text{KL}(\mu_k \parallel \text{sg}(\bar{\mu})),$$

where $\bar{\mu} = \frac{1}{K} \sum_k \mu_k$ and `sg` is stop-gradient (`.detach()`). The first two terms are permutation-invariant in the heads, so the KL-from-mean diversity term is required to break head symmetry and let the heads occupy distinct Θ -consistent modes; the test-time prediction is the mixture marginal $\bar{\mu}_\pi = \sum_k \pi_k \mu_k$.

BEARS. K separately trained UCI encoders ([24]; `train_bears`); member k minimizes its semantic loss plus a KL to the running average of members $0..k$ and a Bernoulli-entropy term, with members $0..k-1$ frozen. We report both the ensemble mean and the best-of- K oracle.

Read as ablations, the baselines isolate each mechanism: the gap of **WMC** over **Independent** is the contribution of weighted model counting over the compiled Θ (circuit present vs. absent), while the gap of **anchored** over **JustWMC** is the contribution of the SDD-compiled support over a learned mixture support (support compiled vs. learned). The JustWMC construction is the empirical face of a result we verify in Lean (§J): every reasoning-shortcut mixture over a *SRIOQ* query is representable by a categorical mixture of UCIs, whose components are indexed by the query’s justifications [17].

H Experimental details

All drivers compile the SDD once, then train each method with Adam; every reported metric is computed on a held-out evaluation split of *unseen* individuals (fresh image pairs and pizzas) disjoint from the training examples, so the numbers measure generalization to new query individuals rather than memorization (sizes per experiment below). The MNIST drivers score real 28×28 images with a CNN (learning rate 10^{-3} , batch 128); the Pizzaiolo fragment trains a linear head on a frozen ResNet-18 (learning rate 10^{-3} , batch 64); the synthetic Pizza control reads a 12-bit feature vector with per-bit flip noise 0.05 through a small MLP (learning rate 10^{-2} , batch 32). Table 6 collects the hyperparameters; the headline numbers below are the committed reference runs (**results/** in the code repository) reported in §4.

All reference runs were produced on a single NVIDIA GeForce RTX 4090 (24 GB) under Slurm; no experiment needs more than one GPU. Compilation is cheap relative to training: the 114-atom MNIST-*SRIOQ* circuit compiles to

Table 6. Hyperparameters (Adam; learning rates and batch sizes in the text). λ : semantic-loss weight; κ : diversity weight; K : mixture / ensemble size.

experiment	feat. dim	hidden epochs	λ	κ	K
MNIST (CNN)	28^2	CNN	12 0.5	–	–
Pizzaiolo frag. (ResNet-18)	224^2	linear head	5 0.5	–	–
Pizza (synthetic)	12	64	10 0.3	–	–
MNIST-Disj. (CNN)	28^2	CNN	20 0.5	2.0	4

Table 7. Pizzaiolo two-topping role-based fragment (mean $\pm$ std over ten seeds; metrics in §4). *topping* is supervised; *property* are the latent Vegetarian/NonVegetarian/Spicy classes recovered through `hasTopping`.

method	topping	property	viol $\downarrow$
Independent	$0.95\pm.01$	$0.45\pm.15$	$0.89\pm.18$
WMC	$0.93\pm.01$	$0.92\pm.02$	$0.06\pm.02$

an SDD of $(7.3\text{--}9.7)\cdot 10^3$ nodes in 1–3 s on CPU (the spread across runs comes from PySDD’s dynamic vtree minimization), the MNIST-Disjunction pair to 10,111 (digit) and 2,972 (gender) nodes, the Pizzaiolo fragment to 670–820 nodes (59 atoms, ~ 2 s), and the synthetic Pizza control to $\sim 1.2 \cdot 10^5$ nodes. Training wall-clock per method and seed: MNIST-*SRQIQ* 68 s for WMC (8 epochs) and 2 s for Independent; the two-regime RS driver 108 s per WMC regime (12 epochs); the Pizzaiolo fragment and MNIST-Disjunction each finish in minutes. No per-method hyperparameter tuning was performed: within each table all methods share the optimizer, learning rate, batch size, epoch count, and λ of Table 6, with the mixture-specific (κ, K) fixed once across methods and seeds, so no baseline received a smaller tuning budget than BAOBAB.

For the synthetic Pizza-*SRQIQ* control (Table 9; the single-individual precursor to Pizzaiolo-*SRQIQ*, §4.1), each example is a 12-bit topping vector (noisy); the latent atoms are the four named pizzas and seven derived classes (`Pizza`, `NamedPizza`, `MeatyPizza`, `CheeseyPizza`, `RealItalianPizza`, `InterestingPizza`, `NonVegetarianPizza`), entailed by the curated 28-concept subset. Supervision reveals 50% of the *topping* atoms only; the class atoms are never supervised. Train / eval = 128/64. The compiled circuit has 66 atoms and an SDD of $\sim 5.3 \cdot 10^4$ nodes. Independent recovers the named-pizza class near chance (0.44, latent 0.43) and violates the ontology on every example (rate 1.00); the WMC loss lifts the class to 0.75 (latent 0.58) and drives violations to 0.00 (mean over ten seeds).

MNIST-*SRQIQ* (§4.1) compiles a 52-axiom ontology (`experiments/ontologies/mnist_sroiq.ofn`) over the digit classes 0–4 into a 114-atom SDD ($5.3 \cdot 10^9$ models, <1 s compile). It instantiates every distinctive *SRQIQ* feature: subsumption (each digit $\sqsubseteq$ `Number`, parity, primality); disjointness (the five digits; `Even/Odd`; `Prime/Composite`); the covering disjunctions `Number` $\sqsubseteq$ `Even` $\sqcup$ `Odd` and `Number` $\sqsubseteq$ $\bigsqcup_d d$; the complements `Odd` $\equiv$ `Number` $\sqcap$ $\neg$ `Even` and `NonPrime` $\equiv$ `Number` $\sqcap$ $\neg$ `Prime`; the universal successor closures `d` $\sqsubseteq$ $\forall \text{succ.}(d+1)$ and `Number` $\sqsubseteq$ $\forall \text{succ. Number}$; the qualified number

Table 8. MNIST-Disjunction ($\#RS = 4$; mean $\pm$ std over ten seeds; metrics in §4). NLL is over the gender atoms (Bayes-optimal 4.16); ECE and mode-coverage TV are the reasoning-shortcut diagnostics. BEARS is a $K=4$ deep-ensemble mitigation [24] trained on the same circuit; its per-digit accuracy is not separately evaluated (–).

Method	digit	NLL ↓	ECE ↓	mode-cov. TV ↓
Independent (UCI)	0.99 $\pm$.00	6.93 $\pm$.00	0.00 $\pm$.00	1.00 $\pm$.00
Single-WMC (SL / DPL)	0.99 $\pm$.00	6.13 $\pm$.00	0.00 $\pm$.00	0.99 $\pm$.00
JustWMC (learned mix)	0.99 $\pm$.00	6.69 $\pm$.43	0.11 $\pm$.03	0.99 $\pm$.01
BEARS	–	6.36 $\pm$ 2.36	0.41 $\pm$.12	0.44 $\pm$.11
JustWMC (anchored)	0.99 $\pm$.00	4.17 $\pm$.00	0.00 $\pm$.00	0.02 $\pm$.00

Table 9. Pizza-*SRIOQ*, 64 held-out examples, mean $\pm$ std over ten seeds (higher is better except for violation). The *pizza* (4 latent) and *latent* (11 atoms) columns are the headline: WMC infers them from topping evidence though those atoms receive no direct supervision.

method	topping (observed)	pizza (4 latent)	latent (11 latent)	violation rate
Independent	0.99 $\pm$.00	0.44 $\pm$.12	0.43 $\pm$.08	1.00
WMC	0.76 $\pm$.01	0.75 $\pm$.00	0.58 $\pm$.05	0.00

restriction $\text{HasSuccessor} \equiv \text{Number} \sqcap \geq 1 \text{ succ.Number}$; the inverse $\text{pred} \equiv \text{succ}^-$; the role hierarchy $\text{succ} \sqsubseteq \text{related, lessThan}$; the chain $\text{succ} \circ \text{succ} \sqsubseteq \text{plusTwo}$; transitive lessThan ; symmetric differsFrom ; asymmetric succ/lessThan ; irreflexive succ/differsFrom ; reflexive sameDigitAs ; and functional succ . Each example is a pair of real MNIST images scored by one `MnistEncoder` CNN; train / eval = 3000/1000, 12 epochs, semantic-loss weight 0.5. The two regimes of Table 1 differ only in the evidence revealed: *grounded* reveals $\text{succ}(a, b)$, `Number`, and a random 50% of the parity/primalty atoms ($\#RS = 1$); *under-determined* reveals only $\text{succ}(a, b)$ and `Number`, leaving the five cyclic relabelings of 0–4 consistent ($\#RS = 5$, computed exactly on the SDD). Revealing one parity atom drops $\#RS$ to 3; the full parity/primalty profile of one slot, to 1. The chain fires only at $|\Delta| = 3$ and is compiled in the companion file `mnist_chain.ofn`.

MNIST-Disjunction (§4.2) puts a real MNIST digit on each of three individuals a, b, c . Because compiling the digit and gender disjunctions jointly over the shared individuals blows up the SDD, we compile two circuits driven by one shared CNN (`run_mnist_disjunction.py`): a 114-atom *digit* circuit (the `mnist_sroiQ` ontology over a, b , with $\text{succ}(a, b)$ coupling their digits) that the CNN recovers through, and a 69-atom *gender* circuit (the disjunction ontology over a, b, c : $\text{Person} \sqsubseteq \text{Male} \sqcup \text{Female}$, $\text{Male} \sqcap \text{Female} \sqsubseteq \perp$, $\text{Male} \sqsubseteq \forall \text{marriedTo.Female}$ and its dual, $\text{hasChild} \equiv \text{hasParent}^- \sqsubseteq \text{hasAncestor}$ transitive) whose four consistent gender modes (a, b opposite; c free) the *image-free* gender heads carry. Supervision reveals the digit parities (recovering the digits) and the structural role/class assertions; the six gender atoms and the four derived role atoms are latent. Train / eval = 2000/800, 20 epochs, $\lambda = 0.5$. The latent NLL is summed over

Table 10. Per-OWL-axiom-kind encoding cost for the MNIST-*SR_QIQ* ontology. *ours*: normalized DL-clauses produced by the structural transformation. *DPL Horn* / *DPL manual*: DPL admits / requires user-written disjunctive / cardinality / negation encoding.

OWL kind	axioms		ours	DPL (Horn)	DPL (manual)	DPL (inexpr.)	DPL clauses
SubClassOf	23	46		15	8	0	30
DisjointClasses	14	28		14	0	0	14
EquivalentClasses	3	12		0	3	0	11
RoleInclusion	2	4		2	0	0	2
AsymmetricRole	2	4		2	0	0	2
IrreflexiveRole	2	4		2	0	0	2
InverseRoles	1	2		1	0	0	2
FunctionalRole	1	2		0	1	0	1
ReflexiveRole	1	2		1	0	0	1
SymmetricRole	1	2		1	0	0	1
TransitiveRole	1	2		1	0	0	1
RoleChain	1	2		1	0	0	1
Total	52	110		40	12	0	68

the ten gender + derived atoms (Bayes-optimal $6 \log 2 \approx 4.16$, the genders contributing $6 \log 2$ and the entailed derived atoms 0). The CNN recovers the digits at 0.99 across all methods; only the *anchored* JustWMC, seeded from the four completions the gender circuit enumerates, reaches the Bayes-optimal NLL 4.17 with mode-coverage TV 0.02 and no seed variance, while the single-WMC and learned-mixture methods stay at TV ≈ 0.99 .

For the size comparison with DeepProbLog (§4.3), running the per-axiom encoding analysis (`baobab/nesy/sroi-deepproblog_iface.py`) on the MNIST-*SR_QIQ* ontology, 12 of its 52 axioms (23%) leave ProbLog’s Horn fragment: the eight universal-restriction and disjunctive subclass closures, the two complement and one qualified-cardinality equivalences, and the functional-role axiom. Under a charitable manual translation none is strictly inexpressible, but each abandons the Horn structure DeepProbLog compiles natively (Table 10).

I Mechanized guarantees

The four results summarized in §3.3; all are machine-checked in the `ELKSDD` and `GroundingRefinements` libraries under `lean/` in the released code, with the module inventory in App. J.

Theorem 2 (Saturation is model-preserving). *The $\mathcal{ALCHOQ}$ saturation is sound, and the saturated grounding is logically equivalent to Θ_Δ ; in particular the two have the same weighted model count.*

Proof. Every clause the saturation derives is entailed by its input (soundness, mechanized in Lean and axiom-free). Adding an entailed clause to a propositional

theory changes neither its models nor the weight assigned to any interpretation, so the saturated grounding and Θ_Δ have the same weighted model count.

Theorem 3 (Faithful grounding on the grounding-covered fragment). *Each grounding rule emits clauses that hold in exactly the interpretations the Tarskian semantics of its construct admits. Consequently Θ_Δ is sound: every $SRQIQ$ interpretation over Δ is a model of Θ_Δ . When every existential consequence is discharged by grounding or is function-free, the converse holds and the models of Θ_Δ are exactly the $SRQIQ$ interpretations over Δ ; otherwise Θ_Δ over-approximates them, the gap being precisely the head-existential consequences whose witnesses the finite grounder cannot name (§3.1).*

Proof. One Tarskian-soundness lemma per grounding rule (App. D), each mechanized in Lean; soundness of Θ_Δ follows since every emitted clause holds in every $SRQIQ$ interpretation over Δ . Exactness on the grounding-covered, function-free fragment holds because there every normalized clause is represented; the residual gap is exactly the Skolem-term clauses the grounder drops.

Theorem 4 (Weighted model count is the semantic target). *On the compiled SDD, $WMC(\Theta \mid p_\theta)$ equals the probability the ontology assigns to its models under the distribution semantics [30], the quantity the losses of §3.2 optimize.*

Proof. The bottom-up evaluation of the SDD computes the world-sum of the distribution semantics; the equality is mechanized in Lean (App. E).

Theorem 5 (RS-awareness for $SRQIQ$, after [17]). *A single independent (UCI) perception cannot represent a reasoning-shortcut mixture unless its valid completions form one justification’s subcube, whereas a mixture of UCIs indexed by the query’s justifications represents every such mixture.*

Proof. The two directions are the necessary and sufficient conditions of [17]; we instantiate their implicants as the query’s justifications and mechanize both directions for $SRQIQ$.

The development is foundation-only: every audited theorem reports just `{propext, Classical.choice, Quot.sound}` under `#print axioms` (the $ALCHOQ$ saturation soundness is axiom-free), with no `sorry`s and no added axioms.

J Lean module inventory

The Lean 4 development is structured as follows. `ALC.lean` carries the ALC syntax, Tarskian semantics, the `eval_neg_*` duality lemmas, and a sound CB calculus with the `monoExist/monoUniv` role-axis monotonicity rules. `ALCHOQ.lean` adds nominals (`Concept.nom`) and qualified number restrictions (`Concept.atLeast`, `Concept.atMost`), with cardinality predicates `atLeastCard/atMostCard` and the filler-monotonicity rules `monoAtLeast` (covariant) and `monoAtMost` (contravariant); the soundness theorem `ALCHOQ.sat_sound` reports zero axioms. `SRQIQ.lean`

defines the `RAxiom` inductive (`incl`, `chain`, `trans`, `sym`, `asym`, `refl`, `irrefl`, `inv`, `disj`) with a Tarskian `RAxiom.eval` and a soundness lemma per shape (`incl_sound`, `trans_sound`, `sym_sound`, `asym_sound`, `refl_sound`, `irrefl_sound`, `inv_sound`, `disj_sound`, `chain_two_sound`). Role identities `trans_iff_chain` and `sym_iff_self_inverse` justify the transitivity-as-chain and symmetric-as-inverse shortcuts; `has_self_iff` justifies the local-reflexivity proxy grounding rule.

Completeness goes via a canonical model. `Completeness.lean` mechanizes $\mathcal{ALC}$ completeness end-to-end: a strictly stronger calculus `ALC.SatC` extends `ALC.Sat` with the classical-logic rules necessary for completeness: double-negation, excluded middle, non-contradiction, de Morgan, $\exists/\forall$ -duality, the join rule $\exists R.C \sqcap \forall R.D \sqsubseteq \exists R.(C \sqcap D)$, Boolean distribution, and the role-axis closures $\exists R.\perp \sqsubseteq \perp$ and $\top \sqsubseteq \forall R.\top$. The headline theorem `satC_complete` : $\mathcal{O} \models C \sqsubseteq D \rightarrow \text{SatC } \mathcal{O} C D$ is fully proved via a Lindenbaum canonical model: `lindenbaum` uses Mathlib's `zorn_subset_nonempty` on `consistent_chain_union`; `lindenbaum_max_closed` follows by case analysis on excluded middle and Boolean distribution; the truth lemma `canonical_eval_iff` is by induction on `Concept` (propositional cases via `top_mem`, `bot_not_mem`, `mem_xor_neg`; role-axis cases via `witness_exist` from the iterated $\exists R$ -join lemma `satC_exist_with_univs`, `satC_map_univ_to_univListConj`, and `existBot`). `ALCHOQCompleteness.lean` lifts every $\mathcal{ALC}$ -classical rule to $\mathcal{ALCHOQ}$ via `ofAlchoq`, plus the nominal identity `nomRefl` and the cardinality boundary closures $\top \sqsubseteq \geq 0 R.C$, $\leq 0 R.C \sqsubseteq \forall R.\neg C$, $\forall R.\neg C \sqsubseteq \leq 0 R.C$. Completeness for $\mathcal{ALCHOQ}$ is stated as a conjecture and deferred: closing it requires the auxiliary-individual construction of [33].

`SROIQCompleteness.lean` adds the role-axis consequence rules made available by membership of an `RAxiom`: role inclusion yields $\exists r.C \sqsubseteq \exists s.C$ and $\forall s.C \sqsubseteq \forall r.C$; binary chains $r_1 \circ r_2 \sqsubseteq s$ yield $\exists r_1.\exists r_2.C \sqsubseteq \exists s.C$ (and a k -ary generalization via `existChain_eval_iff`); transitivity yields $\exists r.\exists r.C \sqsubseteq \exists r.C$; reflexivity yields $C \sqsubseteq \exists r.C$ and $\forall r.C \sqsubseteq C$; irreflexivity and role disjointness yield $\exists r.\{i\} \sqcap \{i\} \sqsubseteq \perp$ and $\exists r.\{i\} \sqcap \exists s.\{i\} \sqsubseteq \perp$. Soundness `satC_sound` is fully proved; completeness for *full* $\mathcal{SROIQ}$ is left as the conjecture `sroiq_complete_conjecture`. A canonical-model construction over $\mathcal{SROIQ}$ -side maximal-consistent types is given in `SROIQCanonical.lean`; `SROIQSkolemCanonical.lean` delivers *unconditional* $\mathcal{SROIQ}$ completeness on a `SkolFragment` ontology shape for the role-axiom sub-families (role-inclusion only; role-inclusion + reflexivity; role-inclusion + reflexivity + irreflexivity), and conditional completeness (parameterized on a canonical-RBox- satisfaction witness) for the remaining shapes.

Turning to the Tena Cucala context-structure calculus, `ALCHOIQContext.lean` encodes the core definitions of [33]: Σ_u , context a- and p-terms, context clauses, admissible orders, context structure $D = \langle V, E, \text{core}, S, m, \theta \rangle$, trigger sets S_u, P_r, S_u^r, P_r^r , expansion strategies, soundness, and derivations. The calculus's twelve inference rules (Core, Hyper, Eq, Ineq, Factor, Elim, Join, Nom, Succ, Pred, r-Succ, r-Pred) each have a concrete refinement (`StepCore`, `StepHyper`, ...) with a per-rule soundness lemma. Two unified meta-soundness lemmas (`mono_ext_sound`, `mono_restr_sound`) and a common `StepAddEntailed` schema streamline the

proofs; the edge-adding rules require explicit well-formedness preconditions and full case-analysis on whether each edge is new or pre-existing. The completeness statement of [33] is exposed as a typed Prop `TenaCucalaCompleteness`; each ingredient (model fragment R_i^* , nominal naming, composite Herbrand model, refutation lemma) is a separate structure / definition with real semantic Props. An unconditional sliver (a Bool-Herbrand refutation construction) delivers Tena Cucala completeness for the empty ontology on a propositionally-refutable fragment of queries (`tenaCucalaCompleteness.empty0.propRefutable`).

On complexity, `ALCCComplexity.lean` formalizes the polynomial-degree grounding bounds: `size` measures syntactic concept size, `subconcepts_card_le_size` bounds the subconcept Finset, `saturation_pair_count_le` and `alc_subsumption_pair_bound` give the $|\text{Sat}(\mathcal{O})| \leq |\Sigma|^2$ bound, and the per-feature bounds (`at_least_grounding_size_bound`: $\geq n \cdot R.C$ contributes $4n \cdot d^{n+1}$ literals; chains contribute d^{k+1} tuples; transitivity, hierarchy, inverses contribute d^2 or d^3) compose into `grounding_size_polynomial`.

On the optional grounding refinements, `GroundingRefinements.lean` mechanizes refinements (i) and (ii) of §D as model-count-preservation lemmas over the weighted model count `WMC` (a Boolean model predicate weighted by per-atom literal weights). The workhorse `wmc_split` proves that `WMC` factorizes over an independent atom partition $\text{Core} \oplus \text{Extra}$, and `wmc_unique` (hence `wmc_factor`) proves that an `Extra` block with a unique satisfying assignment contributes a single constant factor. Refinement (i) is `wmc_prune_forcedFalse` (the pruned atoms' only model is all-false) and (ii) is `wmc_skip_equality` (the equality block's unique-name model); `wmc_card_preserved` gives exact model-count preservation under unit weights, and `wmc_log_offset` shows the $-\log \text{WMC}$ objective shifts by a constant whose gradient in the perception weights is zero.

Every theorem in every module passes the foundation-only axiom audit: `#print axioms` reports only `{propext, Classical.choice, Quot.sound}`, with no `sorry` and no user-introduced axioms.